HONO CONFERENCE 2026

30分、お付き合いしてネ😉

開発者チャン💕 Honoを使お😘

田中 博悠 / Hirohisa Tanaka
@tanahiro2010

田中博悠ですヨ〜😊

高校1年生なんだヨ😅

- 三田学園高等学校に通ってるヨ🏫
- Hono Conference 2026のコアスタッフです😊
- GDG Greater Kwansaiにもいるヨ✨
- Alpha+ Projectの4期生なんだナ😉
- 個人開発も、インターンも、競プロも💪
- 音楽とバンジーも好きなんだヨ🎵


バンジージャンプをする田中博悠

学生ダヨ / 作るヨ / 話すヨ

Hono ≒ バンジージャンプの数学的検証だヨ〜

発表しようとしたら別のプロポーザルが採択されたんだナ😅

なんとネ 66%成立😳

Hono ≒ バンジー、なんだナ😉

ところが、ドッコイ‼️😅

その話は、しないヨ〜😅

プロポーザル、採択されなかったんだナ💦

気になったら、懇親会で声かけてネ😉

好きな フレームワーク ありますか？

- **Hono**: 今日の主役
- **React**: UIを作るときの選択肢
- **Ruby on Rails**: 育てながら作れる一体型

- 好きな技術と、必要な技術は同じとは限らない
- 「あとで使うかも」を先に実装してしまう

でも技術選定には
何度も失敗しました

アンケートの結果 個人開発では...

- **Next.js**: 大きなReactエコシステム
- **Nuxt**: Vueのフルスタック体験
- **TanStack Start**: 型付きルートとServer Functions
- **React SPA**: 自由なクライアント構成
- **Rails**: 成熟した一体型の開発体験
- **Hono**: Web標準に近い小さな入口

> 選択肢の多さではなく、必要な複雑さの量で選ぶ

個人開発の敵は**複雑さ**

- 境界が増える
- 状態を同期する
- 実行場所(デプロイ先)を考える
- **お金**

「あとで必要かも」

最初から背負っていませんか？

- **境界**: フロントエンドとAPIを分ける
- **同期**: 状態とデータ取得を揃える
- **実行**: hydrationとブラウザJSを管理する
- **性能**: キャッシュ戦略を考える
- **配備**: 複数のデプロイ先を扱う
- **運用**: 増えていく手数を引き受ける

増やす前に考える

便利な仕組みを必要になる前から持っていないか

小説投稿サイトを没にしました

01 / FETCH

ブラウザから
大量のAPIを呼ぶ

→

02 / WAIT

アプリとDBの
配置場所も未整理

→

03 / DROP

待ち時間を改善できず
開発を中止

当時の構成を見てみます。

- ブラウザで取得してから画面を組み立てる
- API・アプリ・DBの距離をあとから考える

当時はNext.js + Vercel + Neon

Next.js	画面とAPI Routesを実装
Vercel	Next.jsアプリの公開先
Neon	データを保存するPostgreSQL

ブラウザ → Vercel上のAPI → NeonのDB

useEffectでデータを取得し、ブラウザで画面を組み立てる。
アプリとDBのリージョン（配置地域）も未整理でした。

> Next.jsが悪いという話ではありません。
> 当時の自分の実装と構成の問題です。

非同期処理とリージョンを十分に設計せず、原因を測る前に複雑さだけ増やしていました。

初期表示
最悪で約1分。

言い過ぎかもしれないけど少なくとも30秒以上はした

ただの単位計算機なのに
技術スタックは、Next.js

計算する		送る		待つ	収益
ブラウザで十分	→	値をバックエンドへ	→	ロードが重い	どこぞのAdsence

- 計算はブラウザで完結できる
- 毎回APIへ送る理由がない
- 待ち時間だけが増えていく

広告収益化まで考えると、無料プランの利用条件も気になります。

LPなのにReact SPA

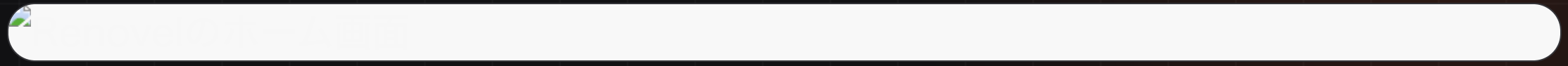
ページは複数。ブログもある。でもmetaはいつも同じ

- ページは複数ある
- ブログもある
- でもmetaはページごとに変わらない

ページごとの情報

それ、Honoでいいじゃん

そこでReNovelを作り直しました



- 投稿 / 閲覧 / 認証 / 検索
- 本棚 / いいね / 通知
- 見た目は、Honoだから制限されない

この見た目はHonoで動いています

割と中規模なアプリを React / Next.jsなし で動かす

- React / Next.jsなし
- HTML / Hono JSX中心で育てられる

SX VIEW / 9 ROUTE MODULE

構成はまずこれだけ

- **Runtime / Web**: Bun + Hono
- **Database / ORM**: PostgreSQL + Drizzle
- **View**: Hono JSX + Kiwa UI

CHAPTER 02 / 必要になってから複雑にしよう

複雑さは
あとで払おうネ😘

最初からAPIとかゴリゴリに設計しなくても、
個人開発だからこそまずは〝単純〟に始めれる

- まずHTMLを返す
- 必要になったらフォームを強化する
- さらに必要なら、その部分だけJSを足す

まずはHTMLを返せばいい

GET /novel		APPLICATION		HONO JSX		BROWSER
URL + HTTP	→	Use case	→	HTML	→	HTML

- 初期表示はサーバーで完了(SSG)
- ブラウザ側の仕事を増やさない

初期表示のために APIを作りすぎない

- SSGだからこそ、HTMLで返せるものはHTMLで返す
- APIは、APIが必要なところに使う

`hono/jsx` でHTMLを返す

JSXを書いて、サーバーでHTMLを作る

```tsx
/** @jsxImportSource hono/jsx */
app.get('/novel', (c) => c.html(
  <NovelPage novel={novel} />
))
```

- JSXを書く
- サーバーでHTMLを作る
- ブラウザには完成したページを返す

一つのアプリで責務は混ぜない

画面	route / controller / JSX view
処理	use case / query / service
データ	Drizzle / PostgreSQL

- 最初からサービスを分割しない
- でも、画面・処理・データは混ぜない
- 必要になった境界だけ、あとから深くする

一つのアプリで育てる

ページ全体はSSR。動くのは一部分

- **SSR**: ページ全体をHTMLで返す
- **Forms**: リンクとHTMLフォームで動かす
- **Island**: 操作する部分だけJSを足す
- ページ全体: SSR
- 遷移や送信: HTML
- 局所的な操作: Island

islandはHTMLの中の小さな島

HTMLの海に浮かぶ、小さなインタラクション

- HTMLの海に浮かぶ小さなUI
- ページ全体は動かさない
- 必要な場所だけブラウザで実行する

まずHTML / 必要なJS

`hono/jsx/dom` でそこだけ動かす

ページ全体を動かさない。
必要な部分だけ動かす

```tsx
/** @jsxImportSource hono/jsx/dom */
export function LikeButton() {
  return <button onClick={toggle}>❤️</button>
}
```

- 全体をhydrationしない
- ボタン単位でJavaScriptを送る
- 操作する場所だけ状態を持つ

動かしたいところだけJSを足す

動かしたい場所だけ、islandにする

いいね　/　フォロー　/　評価　/　本棚追加　/　本棚削除

- 本文の表示: サーバー
- ページ遷移: HTML
- 局所的な操作: JavaScript

ReNovelで実際に使った場所

まずはJavaScriptなしで動かす

- JavaScriptなしでもPOSTできる
- 失敗しても基本の操作は残る
- JavaScriptは操作感のために足す

```
<span
  data-island="like"
  data-props={JSON.stringify({ action, liked, count })}
>
  <form method="post" action={action}>
    <button type="submit">❤️ {count}</button>
  </form>
</span>
```

- 状態をローカルで更新する

必要な場所だけブラウザで強化する

```tsx
/** @jsxImportSource hono/jsx/dom */
import { useState } from 'hono/jsx'

export function LikeButton({ action, liked: initial, count: initialCount }) {
  const [liked, setLiked] = useState(initial)
  const [count, setCount] = useState(initialCount)

  async function toggle() {
    const next = !liked
    setLiked(next)
    setCount((value) => value + (next ? 1 : -1))
    await fetch(action, { method: 'POST' })
  }

  return <button onClick={toggle}>♥ {count}</button>
}
```

ブラウザに送るJSを絞る

34750B

islands bundle

圧縮前

13301B

islands bundle

gzip

- 圧縮前: 34,750 bytes
- gzip: 13,301 bytes
- 動く部分だけを送る

測った結果

サーバー側も測ってみる

0.33MB

server bundle

Bun minify

3.2ms

home TTFB

ウォーム中央値

2026-09-23 ローカル開発環境で計測 / 初回home TTFB 117.7ms / 数値は登壇直前に再計測します

- ローカル開発環境での計測
- ウォーム中央値: 3.2ms
- 本番性能の保証ではない

SEOは最初のHTMLから始まる

各URLの情報が、最初のHTMLに入っているか

- 本文
- title / description
- canonical / OGP

最初のHTMLに何を入れる？

- **Content**: 本文 / title / description
- **Identity**: canonical / OGP
- **Delivery**: JS実行を待たずに返す

ReactやNext.jsでもSSRで実現できます。
Honoではページ全体のhydrationを前提にせず選べます

- フレームワーク名ではなく、返すHTMLを見る
- 各URLの情報を各URLで持つ
- 必要なJSだけを選ぶ

Honoを小さく公開する

APP		WORKERS		SHIP
Hono appをexport	→	エッジでリクエストを処理	→	`wrangler deploy`

まずはアプリを小さく公開する

- Hono appをexport
- `wrangler deploy` で公開
- 公開までの距離を短くする

データが必要ならD1につなぐ

REQUEST		DATA		RESPONSE
Hono route	→	D1 binding	→	HTML / JSON

Renovel本体はBun + PostgreSQLです。Workers + D1は持ち帰り用の別サンプルです。

- Renovel本体: Bun + PostgreSQL
- ここで紹介する構成: Workers + D1
- 別の選択肢として考える

個人開発のデプロイ先どうする？

Next.js		Hono
Vercel / AWS など	→	Cloudflare Workers

小さく始めて、収益化の余地も残す

Vercel Hobbyは商用利用に制限があります。Cloudflare Workersには無料枠があります。プランと利用条件は公開前に確認します。

- 料金だけで決めない
- 商用利用の条件を確認する
- 無料枠・制限は公開前に確認する

Cloudflareであとから払える基盤を作る

FREE
無料枠から始める

→

EDGE
世界のエッジで動かす

→

PAID
必要になったら払う

だからHonoは Cloudflareと相性がいい

- Web標準に近い
- 小さく公開しやすい
- 必要になったら構成を広げる

どれが上かではなく必要なものを見る

- **Next.js**
  Reactの大きなエコシステム、RSC、Server Actions、画像最適化を使いたいとき
- **TanStack Start**
  型付きroute tree、loader、client navigation、Server Functionsが欲しいとき
- **Hono**
  SSR中心で、局所的な操作とWeb標準に近い実装をしたいとき

今の自分に必要なものを選びます。

Honoが合うのはこんな個人開発

合いやすい

- CRUDが中心で画面の動きは局所的
- APIとSSRを同じ言語・同じアプリで書きたい
- Reactの巨大なエコシステムは必須ではない
- 運用コストと認知負荷を抑えたい

Renovelでの判断

こういうときは別の選択肢

別の選択肢も考える

- FigmaやCanvaのように画面全体が動く
- 大規模なクライアント状態が中心
- チームが別のフレームワークへ習熟している
- その機能群自体がプロダクトの強みになる

CHAPTER 03 / まず小さく作ってみる

小さく作ろうネ😊
必要な分だけ育てよ🌱

03

まずはサーバーだけで動かしてみる

01

まずHTMLを返す

02

まずフォームで動かす

最初に返すもの

必要になったらそこだけ足す

03

必要な場所だけJSを足す

04

そのまま公開する

必要な分だけ育てる

Honoを使え、
という話ではありません

個人開発者よ
複雑さをあとで払える
道具として
Honoを使え。